



AutoDQ: A Configurable DSL Framework for Enterprise Data Quality Validation

ANEERBAN CHOWDHURY, M24DE3011

MTECH IN DATA ENGINEERING, IIT JODHPUR

SUPERVISOR: DR. GOURHARI GHOSH

Abstract

In modern data ecosystems, ensuring the quality, integrity, and compliance of data is paramount. However, defining and validating data quality (DQ) rules across diverse pipelines remains largely ad-hoc and hardcoded. This project introduces a **declarative, domain-agnostic, and extensible data quality framework** driven by a **custom, configurable and scalable YAML/JSON-based DSL, i.e. Domain-Specific Language**.

The framework allows users to define expressive DQ rules without programming, and supports validations such as schema constraints, inter-field logic, pattern matching, outlier checks, and temporal validations. A modular rule execution engine interprets the DSL and applies the rules across varied data sources.

Additionally, a profiling-based auto-rule suggestion module aids rapid onboarding for new datasets, and a lightweight reporting interface provides summaries, logs, and DQ health scores. The system is designed to be extendable as a plug-in for orchestration tools like Airflow or dbt. The resulting solution significantly reduces time-to-insight, enforces governance at scale, and empowers technical and business users to collaboratively manage data quality.

Motivation

85% of data science projects fail due to poor data quality.

Business teams lack tools to define rules easily.

Developers struggle to enforce reusable validation rules in pipelines.

Need for a DSL that's simple, expressive, and execution-agnostic.

Several modern data quality tools—such as **Amazon's Deequ** have introduced validation frameworks. However, these tools exhibit key limitations in terms of rule flexibility, language expressiveness, integration with orchestrators and cross-environment support (e.g., SQL, Pandas, Spark). They also often require programming knowledge, making them inaccessible to non-technical stakeholders.

Problem Statement

Existing approaches to data validation are either tightly coupled with specific execution engines, require verbose configurations, or lack flexibility in defining inter-field and temporal quality rules. This creates a significant barrier in scalable and repeatable quality enforcement, especially in cross-functional teams dealing with diverse data formats.

Problem: There is a lack of a lightweight, extensible, and declarative framework that allows both technical and non-technical users to define and apply **complex, multi-layered data quality rules** seamlessly across batch and streaming pipelines.

Existing tools are either too rigid, too technical, or engine-locked and lack of readable, declarative systems for data quality validation creates difficulties and delays in understanding underlying data patterns and issues. The solution it needs is to support cross-field rules, conditional logic, and multi-engine support.

Objectives

- To design a DSL schema (using YAML/JSON) to define data quality rules in a declarative manner.
- To implement a modular rule execution engine that parses these rules and applies them to datasets in various formats (CSV, Parquet, SQL tables, etc.).
- To support advanced rule logic, including cross-column dependencies, conditional logic, regex checks, and temporal constraints.
- To develop an optional profiling module that recommends initial rules based on schema and statistical profiling.
- To create a reporting layer that summarizes rule violations, quality scores, and trends.
- To ensure extensibility via integration with tools like Apache Airflow, dbt, or command-line pipelines.

Literature Survey

Title	Source	Authors / Year	Contribution
Deequ: Declarative Data Quality Checks at Scale	ACM SIGMOD	Schelter et al., 2019 (Amazon)	Introduced Deequ — a Scala library for defining and running DQ checks declaratively on Spark. Pioneered the idea of metrics + constraints DSL.
A Data Quality Management Framework for Big Data Pipelines	IEEE Access	Barakat et al., 2021	Proposed a layered DQ architecture with profiling, constraints, anomaly detection — emphasizes modular validation layers.
Detecting Data Quality Rules Using Association Rule Mining	Springer	Jagadish et al., 2020	Presents methods to auto-generate rules using pattern mining — aligns with our optional auto-suggestion module.
Data Contracts: Enabling Data Quality at Scale	ArXiv (Meta)	Chen et al., 2022	Discusses “data contracts” as a declarative agreement between producers and consumers to enforce quality. A key motivator for rule DSLs.
A Survey of Data Quality in Data Lakes	Elsevier	Sawadogo et al., 2020	Covers dimensions of data quality in unstructured environments and highlights need for automated schema/profile checks.
Declarative Data Cleaning: Language and Architecture	VLDB / ACM	Chu et al., 2016	Introduced REX and NADEEF for cleaning with rules in a declarative style. Foundational for your DSL ideas.

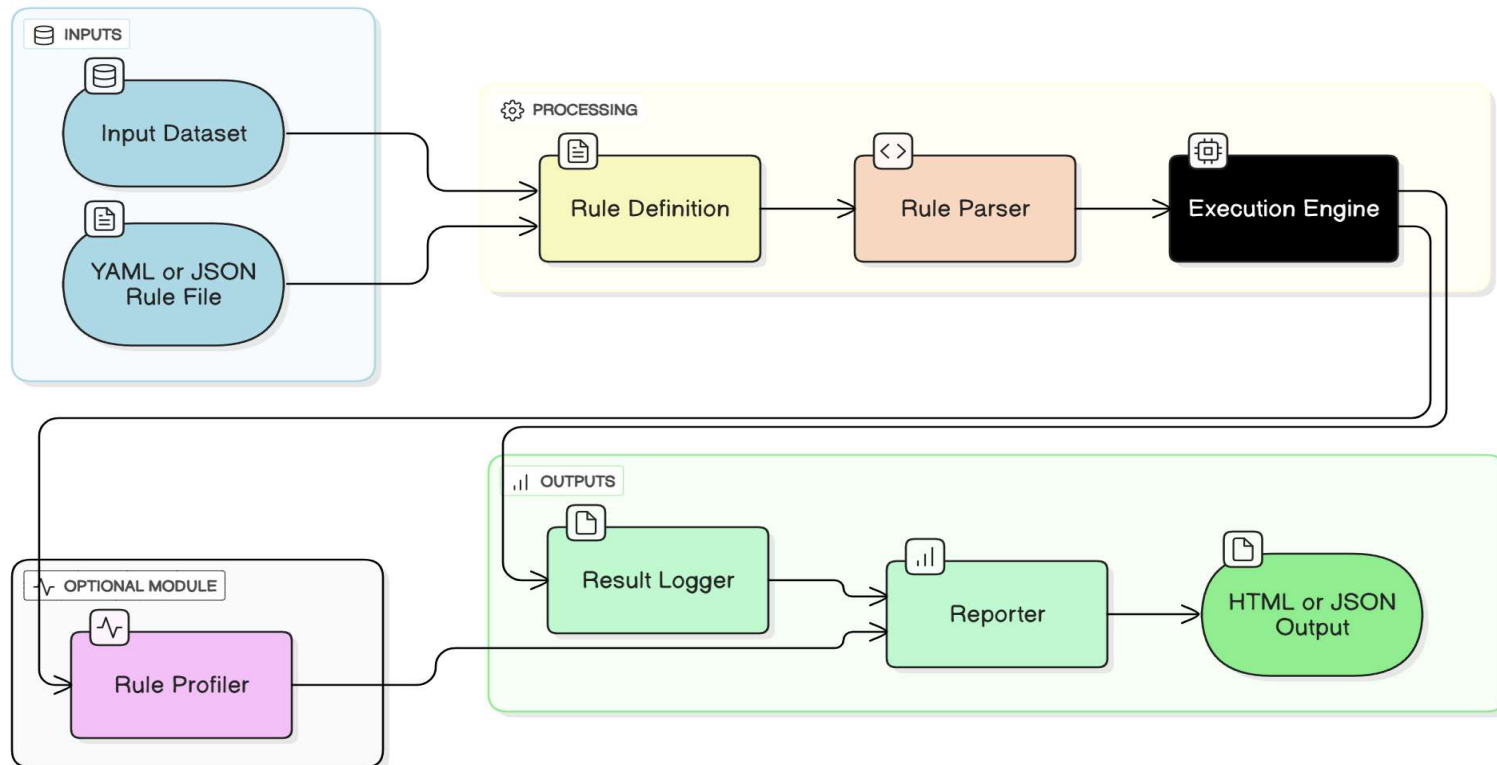
Gaps in Current Tools

Tool	Limitation	Explanation
Deequ (by Amazon)	Only works with Spark + Scala	No support for Python or SQL engines; limits adoption in teams using Pandas or RDBMS.
	Limited rule logic	Doesn't support complex inter-column rules like "If age < 18 then income must be null."
	Not very user-friendly	Rule creation requires some code or builder pattern — not simple YAML.
Great Expectations (GE)	No support for conditional logic	You can't easily write logic like "if A then B" or complex multi-column checks.
	Verbose and rigid YAML syntax	Each check becomes lengthy and specific; poor readability for large sets of rules.
	Limited for integration in orchestrated pipelines	Doesn't naturally plug into Airflow, dbt, etc., unless wrapped manually.

How AutoDQ Fixes These Gaps

Feature	What You Add	Why It's Better
Generic Engine	Supports Pandas, SQLAlchemy, PySpark	Wider adoption across teams and stack types.
Rich DSL	Expressive, readable, concise YAML syntax	Easier for PMs and analysts to define rules.
Cross-column & temporal logic	Support rules like IF condition THEN expectation	Fills the biggest functional gap in GE/Deequ.
Modular	CLI, API, UI, orchestration plug-ins	Enables smooth integration in production workflows.
Optional Auto-suggestions	Profiling-based YAML template generation	Accelerates onboarding for new datasets.

System Architecture



DSL Syntax Example

#1. KYC Type must be valid

- name: Valid KYC Type
column: kyc_type
rule: one_of
value: ["Aadhar", "Driving License", "Voter ID", "Passport"]

#2. Expiry date is required for DL and Passport, must be empty for others

- name: Expiry Required for DL/Passport
rule: conditional
expression: |
if kyc_type in ['Driving License', 'Passport'] then kyc_expiry is not null
else kyc_expiry is null

Conclusion

The framework enables users to define data quality rules in a clear, human-readable format without writing complex code. This lowers the entry barrier for analysts, domain experts, and data stewards to participate directly in enforcing data quality standards. The DSL (Domain-Specific Language) offers a compact syntax to express validations with high precision and readability.

It supports a wide range of validation types, including schema checks (like data type enforcement and null restrictions), logical conditions across multiple fields, pattern matching using regular expressions, detection of statistical anomalies, and temporal consistency checks. These rule types cover both structural and semantic aspects of data quality, ensuring comprehensive coverage across domains.

A lightweight, modular rule execution engine interprets these DSL-defined rules and applies them efficiently across different data platforms. Whether working with flat files, relational databases, or data lakes, the engine ensures scalable and consistent rule application, making it suitable for enterprise-grade data validation workflows.

Thankyou

